

```
"""
```

```
app.py – Backend minimal pour le Live Match Tracker  
Protège la clé API-Football (jamais exposée au navigateur) et  
expose deux endpoints simples que l'application frontend peut appeler.
```

```
Lancer en local : uvicorn app:app --reload
```

```
En production (Render, Railway...) : uvicorn app:app --host 0.0.0.0 --port $PORT
```

```
"""
```

```
import os
```

```
import time
```

```
import httpx
```

```
from fastapi import FastAPI, HTTPException
```

```
from fastapi.middleware.cors import CORSMiddleware
```

```
API_KEY = os.environ.get("API_FOOTBALL_KEY") # défini côté serveur, jamais dans
```

```
BASE_URL = "https://v3.football.api-sports.io"
```

```
HEADERS = {"x-apisports-key": API_KEY} if API_KEY else {}
```

```
app = FastAPI(title="Live Match Tracker – Backend")
```

```
# Autorise l'artifact publié (et n'importe quel front) à appeler ce serveur.
```

```
# À restreindre à votre propre domaine une fois l'app en production.
```

```
app.add_middleware(
```

```
    CORSMiddleware,
```

```
    allow_origins=["*"],
```

```
    allow_methods=["GET"],
```

```
    allow_headers=["*"],
```

```
)
```

```
# --- Cache mémoire très simple pour ne pas exploser le quota gratuit (100 req/jo
```

```
_cache: dict[str, tuple[float, object]] = {}
```

```
CACHE_TTL_SECONDS = 90 # ne réinterroge l'API que toutes les 90 secondes par res
```

```
def cached_get(cache_key: str, url: str, params: dict):
```

```
    now = time.time()
```

```
    if cache_key in _cache:
```

```
        ts, data = _cache[cache_key]
```

```
        if now - ts < CACHE_TTL_SECONDS:
```

```
            return data
```

```
    if not API_KEY:
```

```
        raise HTTPException(500, "API_FOOTBALL_KEY manquante côté serveur")
```

```

with httpx.Client(timeout=10) as client:
    resp = client.get(url, headers=HEADERS, params=params)
    if resp.status_code != 200:
        raise HTTPException(resp.status_code, f"Erreur API-Football: {resp.text}")

    data = resp.json()
    _cache[cache_key] = (now, data)
    return data

@app.get("/health")
def health():
    return {"status": "ok", "key_configured": bool(API_KEY)}

@app.get("/fixtures/georgia")
def fixtures_georgia(date: str):
    """
    Renvoie les matchs du jour dont la ligue est en Géorgie.
    date au format YYYY-MM-DD (ex: 2026-09-21)
    """
    data = cached_get(
        f"fixtures:{date}",
        f"{BASE_URL}/fixtures",
        {"date": date, "timezone": "Europe/Paris"},
    )
    fixtures = data.get("response", [])
    georgia_fixtures = [
        f for f in fixtures
        if f.get("league", {}).get("country") == "Georgia"
    ]
    # simplifie la réponse pour le frontend
    result = []
    for f in georgia_fixtures:
        result.append({
            "fixture_id": f["fixture"]["id"],
            "status": f["fixture"]["status"]["short"], # ex: NS, 1H, 2H, FT
            "minute": f["fixture"]["status"]["elapsed"],
            "kickoff": f["fixture"]["date"],
            "league": f["league"]["name"],
            "home": {"name": f["teams"]["home"]["name"], "id": f["teams"]["home"]},
            "away": {"name": f["teams"]["away"]["name"], "id": f["teams"]["away"]},
            "score_home": f["goals"]["home"],
            "score_away": f["goals"]["away"],
        })
    return {"date": date, "matches": result}

```

```

@app.get("/fixtures/{fixture_id}/stats")
def fixture_stats(fixture_id: int):
    """
    Renvoie les statistiques live d'un match (possession, tirs, corners, fautes)
    pour les deux équipes.
    """
    data = cached_get(
        f"stats:{fixture_id}",
        f"{BASE_URL}/fixtures/statistics",
        {"fixture": fixture_id},
    )
    response = data.get("response", [])
    stats = {}
    for team_block in response:
        team_name = team_block["team"]["name"]
        values = {item["type"]: item["value"] for item in team_block["statistics"]}
        stats[team_name] = values
    return {"fixture_id": fixture_id, "stats": stats}

```